

Practical 9: Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition

Implement secure user registration by inserting user details into MySQL using MySQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using `password_hash()`, and display

Key questions

1. Is `password_hash()` used before storing passwords?

Yes, `password_hash()` is used before storing the password.

2. Is duplicate email/username checked?

Yes, duplicate email/username is checked before inserting the user.

3. Are frontend and backend validations consistent?

Yes, frontend and backend validations are kept consistent to ensure valid and secure input.

appropriate messages.

Db.php

```
<?php

$conn = new mysqli("localhost", "root", "", "user_registration");

if ($conn->connect_error) {
    die("Database connection failed: " . $conn->connect_error);
}

?>
```

Register.php

```
<?php

include "db.php";

$message = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $name = trim($_POST["name"]);
```

```
$email = trim($_POST["email"]);
$password = $_POST["password"];
$confirm_password = $_POST["confirm_password"];

// Backend validation
if (empty($name) || empty($email) || empty($password) ||
empty($confirm_password)) {

    $message = "All fields are required.";

} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {

    $message = "Invalid email format.";

} elseif ($password !== $confirm_password) {

    $message = "Passwords do not match.";

} else {

    // Check duplicate email
    $stmt = $conn->prepare("SELECT id FROM users WHERE email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();

    $result = $stmt->get_result();

    if ($result->num_rows > 0) {

        $message = "Email already registered.";

    } else {

        // Hash password
        $hashed_password = password_hash($password, PASSWORD_DEFAULT);

        // Insert user
        $stmt = $conn->prepare(
            "INSERT INTO users (name, email, password) VALUES (?, ?,
?)"
```

```
);

$stmt->bind_param("sss", $name, $email, $hashed_password);

if ($stmt->execute()) {
    $message = "Registration successful.";
} else {
    $message = "Registration failed.";
}

$stmt->close();
}
}

?>

<!DOCTYPE html>
<html>
<head>
    <title>User Registration</title>
</head>

<body>

    <h2>Secure User Registration</h2>

    <?php
    if ($message != "") {
        echo "<p>$message</p>";
    }
    ?>

    <form method="POST">

        <label>Name:</label><br>
        <input type="text" name="name"><br><br>

        <label>Email:</label><br>
        <input type="email" name="email"><br><br>
```

```
<label>Password:</label><br>
<input type="password" name="password"><br><br>

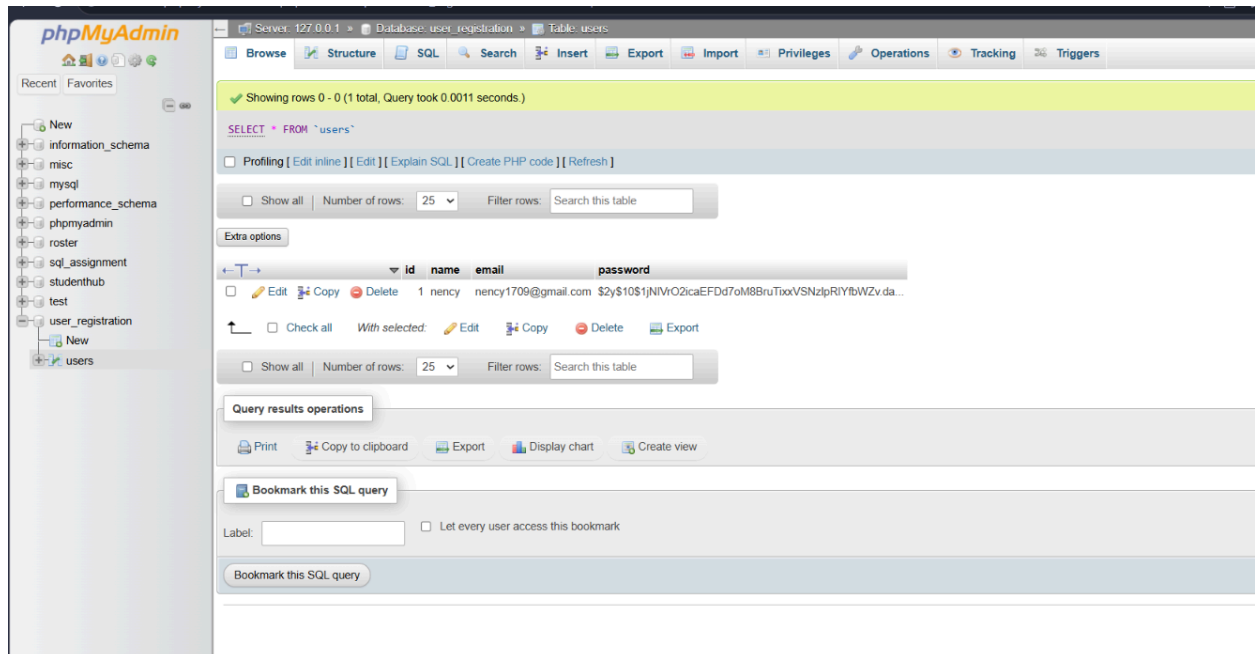
<label>Confirm Password:</label><br>
<input type="password" name="confirm_password"><br><br>

<button type="submit">Register</button>

</form>

</body>
</html>
```

Output



The screenshot shows the phpMyAdmin interface. The left sidebar displays the database structure, including 'information_schema', 'misc', 'mysql', 'performance_schema', 'phpmyadmin', 'roster', 'sql_assignment', 'studenthub', 'test', 'user_registration', and 'users'. The main panel shows the 'users' table with the following data:

id	name	email	password
1	nency	nency1709@gmail.com	\$2y\$10\$1jNivO2icaEFDd7oM8BruTioxVSNzlpRIYbWZv.da...

The interface also includes a 'Query results operations' section with buttons for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'. There is also a 'Bookmark this SQL query' section with a 'Label' input field and a checkbox 'Let every user access this bookmark'.

Secure User Registration

Email already registered.

Name:

Email:

Password:

Confirm Password:

Register

Secure User Registration

All fields are required.

Name:

Email:

Password:

Confirm Password:

Register